



Design Document

Ethan Collier, Brayden Jonsson, Max Petersen, Sathya Tadinada
December 8, 2025

Table of Contents

1	Introduction	3
2	Background	4
2.1	Project Market	4
2.2	Required Technology	4
2.3	Software/Hardware Requirements	5
3	Requirements	7
3.1	System Architecture	7
3.2	Personnel	8
3.3	System Features	9
4	Software Engineering Techniques	12
4.1	Development Process	12
4.2	Tools	12
4.3	Code Review	12
4.4	Documentation	13
4.5	Team Communication and Meetings	13
5	Timeline	14
6	Appendix A: UI Sketches	16
6.1	Gallery Page	16
6.2	Import/Upload Page	16
6.3	Export Page	18
6.4	Admin Page	19
7	Appendix B: Use Cases	20
7.1	Use Case 1	20
7.2	Use Case 2	21
7.3	Use Case 3	22
7.4	Use Case 4	23
7.5	Use Case 5	24
8	Appendix C: Revisions	25
8.1	Introduction	25
8.2	Background	25
8.3	Requirements	25
8.4	Software Engineering Techniques	25
8.5	Timeline	25
8.6	Appendix A: UI Sketches	25
8.7	Appendix B: Use Cases	25

1 Introduction

FreezeTag is a new *free* and open-source image *tagging* software designed for photographers, businesses, or anyone else who appreciates easy-to-use and adaptable image storage. FreezeTag works by running locally on hardware you provide, such as a personal NAS or a computer, and then collects and stores metadata about your photos. In addition, the user is able to provide tags to indicate what groups an image may be part of. For example, a tag could indicate a particular subject, a specific event, or the client the photo was taken for. This data is then stored in a local database, and FreezeTag's easy-to-use interface allows you to search these photos based on the metadata and tags.

The FreezeTag server is also able to load, run, and manage Python-based plugins that customize the functionality of FreezeTag to fit every possible need. FreezeTag ships with some first-party plugins and third-party custom plugins are also supported. Because these plugins are based in Python, this unlocks the powerful artificial intelligence and machine learning capabilities available through modules such as PyTorch and TensorFlow. AI/ML can be used through plugins directly in the app to support facial recognition, automatic captioning/tagging, natural language searching, and more. Plugins can also contribute to a workflow; one first-party plugin supports selecting a group of images, converting them to a specified format, and emailing them to a client. FreezeTag supports concurrency in these plugins so that you don't have to entirely compromise speed for features.

FreezeTag will be designed as a web server, which users will host on their own managed hardware. This generally avoids expensive cloud costs, as most users will have compatible hardware already. The web server will be designed in Go, a language which is fast, garbage-collected, and has strong support for concurrency. The plugins will be written in Python, which is easy-to-use and has strong AI/ML capabilities. Both the servers and plugins will be managed with Docker Compose, which provides some security benefits for protecting against malicious plugins and also allows us to guarantee a consistent environment. The web server will also use a SQLite database for storing metadata and tagging information for photos, which will also be partially accessible to plugins. This entire backend infrastructure will connect to a Next.js frontend, so that the GUI will be designed and usable as if it was a webpage. This has the added benefit of allowing multiple users on a network to easily access the same FreezeTag instance.

2 Background

2.1 Project Market

FreezeTag provides a solution for individuals or businesses that want to store and categorize images without using a cloud-based service. Flexibility through plugins means that FreezeTag can be adapted to even niche use cases with a little configuration and work, and the tagging system allows granular storage and retrieval of very specific images.

Using Python plugins, developers can leverage machine learning libraries like Pytorch and Tensorflow to run their own image classifiers locally, adding another layer to the customizability of the app.

FreezeTag is ideal for freelance photographers, digital artists, web archivists, and especially anyone who already runs their own NAS (a small but very enthusiastic market)

2.1.1 Similar Apps

- DigiKam
 - DigiKam is a local app written in QT that supports similar tagging and plugins.
 - The plugins in DigiKam are the open-source style of adding a DLL to the project (hard to develop).
 - Networked storage is difficult (DigiKam has to be locally installed on each device, and networked drives have to be set up manually)
- Google/Apple Photos
 - Similar image classification tools to ours, but no configuration or customization
 - Easy to use web interface
 - Only on the cloud: no control over your data, sometimes expensive!
 - Folder structure over tag structure: harder to use, less granular

2.2 Required Technology

At a high level, FreezeTag is a pair of servers (backend written in Go, frontend written in Next.js) that communicate back and forth over an API to store and serve images.

2.2.1 List of Leveraged Technologies

- Docker + Docker Compose
- Go Backend

- SQLite
- ImageMagick (for dealing with a range of formats)
- uv Python package manager
- JsonRPC or custom REST over a loopback port for communication between Python and Go
- Gin and Gin-Swagger
- Next.js Frontend
 - Node and pnpm
 - react-dropzone
 - Javascript/undici fetch
 - Jest

2.2.2 List of Technologies to Implement

- Easy Deployment
 - Will be accomplished with a Docker Compose configuration file.
- API Handling on both the frontend and backend
- Image Gallery
- Image Tagging
- Plugin Environment Management
- Plugin Interaction
- Authentication and Admin Management

2.3 Software/Hardware Requirements

Our application is designed from the ground up to have flexible hardware requirements so that it can be used and hosted by a broad range of people and organizations.

2.3.1 Running a FreezeTag Instance

2.3.1.1 Architecture

To run your own FreezeTag instance, all you need is a normal consumer-grade computer running Windows or Linux (or an Intel Mac). Whatever you choose, it has to be capable of running Docker (this includes basically every computer and OS produced recently). During initial development, we'll only support x86 architectures, but in the future we might expand to ARM64 (specifically to run on Apple Silicon).

2.3.1.2 Storage/Bandwidth

If you're going to use FreezeTag as your primary place for storing photos, you might want to use a drive that can store a couple terabytes. There's no requirement on drive speed, but an SSD or a faster hard drive will definitely lead to faster load times.

In the browser interface, we'll display lower resolution high compression thumbnail formats like WEBP, so drive speeds and bandwidth will be as minimal a constraint as possible while you scroll and search your collection.

2.3.1.3 Optional Plugins

The Python plugins supported by FreezeTag can come with their own extra hardware requirements, which is why they're optional! Certain machine learning applications might benefit from a fast CPU or GPU acceleration on top of our basic hardware requirements, and we want to keep the basic hardware requirements as minimal as possible.

2.3.2 Storing Photos on a FreezeTag Instance

To connect to your own (or someone else's) FreezeTag instance, all you need is an internet connection and a modern web browser. The GUI is completely web-based, so there's no particular architecture requirements (it works across all mobile and PC platforms).

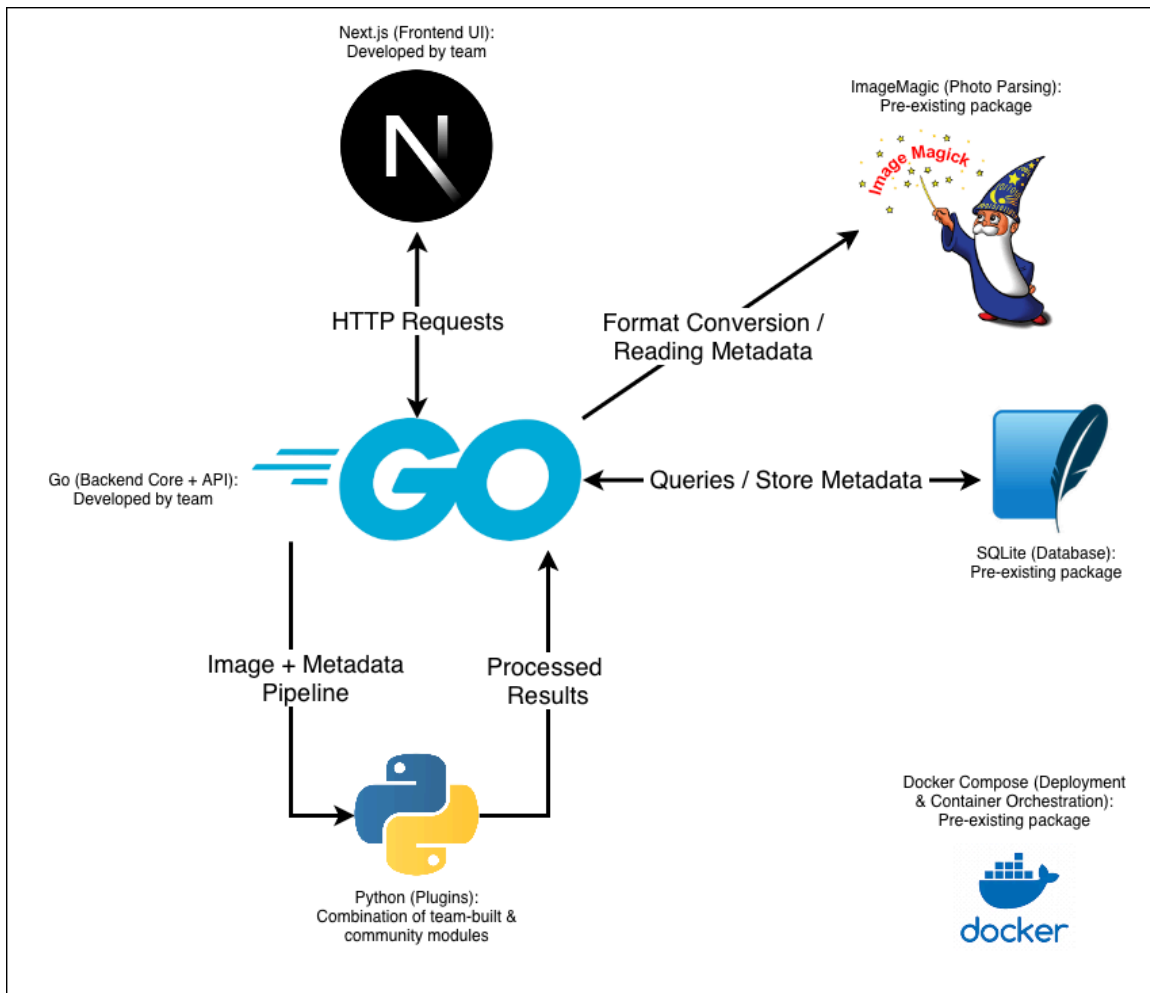
3 Requirements

3.1 System Architecture

The system will be a Go backend server and a Next.js frontend server working together. The Go server will manage plugins, as well as the photo storage and database. The frontend will manage displaying images to the user, accepting uploads from the user, and searching for photos, as well as some authentication (when we add it). The backend will also implement its own authentication in the form of API keys that can be managed from the frontend by an admin user. This way, the frontend can be used by users and the backend can be used by other programs the users might want through the API.

The frontend and backend are both stored in the same monorepo, but in different folders with independent structure. The frontend project structure will utilize Next.js's page routing for the different possible interfaces, but will store common components (buttons, images, etc.) in separate folders. The intent here is that actual pages and their source code will be relatively minimal and focused, with the common components abstracted away.

The backend project structure will be mostly written in Go, utilizing a lot of different Go/C binding libraries to connect to utilities like ImageMagick and SQLite to manage databases and image/metadata parsing. The folder structure will be a pretty standard Go project, but when built it will have a secondary folder for storing Python plugins and their related data, virtual environments, etc.



3.2 Personnel

Brayden will be the frontend “architecture” engineer and Sathya will be the frontend “interface” engineer. The intent here is that Brayden can be in charge of lower-level tasks, such as parsing and handling API responses from the backend, and Sathya can be in charge of actual user interface design and implementation. In practice, it is extremely likely that the frontend will have a lot of mixed development because APIs need UI to be consumed and UI needs APIs to work, especially at the beginning of the project, but we think it’s fair to assign ownership this way. Brayden has direct experience working in a frontend architecture at Lucid, and Sathya has comparatively more experience working on user interfaces. This split also encourages clear separation between user-facing and API-facing parts of the application, which improves separation of concerns in the codebase and makes updates easier.

Likewise, Max will be the backend architecture engineer and will be focusing on tasks such as interacting with plugins and handling containers. Ethan will be the backend “feature” engineer which includes the API and most of the tasks that relate to the actual user-facing features. Max is taking the role as backend architecture engineer due to his experience in the exact type of application management we will need for managing plugins. Ethan likewise has some relevant experience in defining APIs on backend applications.

In addition, both backend engineers will have the primary task of implementing our plugins, but due to the flexibility of plugins the frontend engineers will participate as well when we reach a Rank 3 polish stage. All engineers will be completing at least one “major” plugin when we reach Rank 3.

3.2.1 Primary Assignments

- Sathya, Frontend Interface Engineer
 - Interface Design
 - CSS Implementation
- Brayden, Frontend Architecture Engineer
 - Frontend Data Management
 - Backend Communication
- Ethan, Backend Feature Engineer
 - API Endpoints
 - Primary User Features
 - Image Storage
 - Metadata Parsing
 - Etc.
- Max, Backend Architecture Engineer
 - Dependency Management
 - Plugin Management

3.3 System Features

3.3.1 Rank 1

- Photo Gallery UI
 - A responsive grid-based gallery displaying all uploaded photos.
 - Includes sorting and basic filtering (by upload date, filename, or tag).
 - Each image opens a details modal showing metadata, tags, and captions.
- Uploading UI

- Drag-and-drop or file picker interface to upload images to the server.
- Displays upload progress and confirmation upon completion.
- Uses multipart API requests to transmit file and metadata to the backend.
- Server Connectivity
 - Full client-server integration: frontend served via Next.js, backend serving API requests.
 - Initial deployment supports local testing and remote access (localhost and Docker).
- Tagging and Captioning Database
 - SQLite schema that associates each image with user-defined tags and captions.
 - Supports exact-match search and partial text queries.
- EXIF Metadata Parsing
 - Automatically extracts embedded metadata from uploaded images.
 - Parses fields such as timestamp, camera model, and GPS coordinates.
 - Stores extracted metadata in the database as searchable fields.

3.3.2 Rank 2

- First-Party Plugins
 - Facial Recognition Auto-Tagging
 - Uses a local machine learning model to detect faces and auto-tag known individuals.
 - Local ML Image Captioning
 - Integrates a lightweight image-captioning model for automatic caption generation.
 - Runs offline and stores captions in the metadata database.
 - Location Tagging via OpenStreetMap
 - Uses EXIF GPS data to generate approximate location tags (e.g., nearest city or country).
 - Bulk Exporting Tools
 - Allow users to export selected images and metadata in common formats (ZIP, TAR, or email attachments).
- Community Plugin API
 - Exposes a public API endpoint for developers to submit and manage community plugins.
- Docker Deployment Support
 - Provides a Docker Compose file that automatically sets up the backend, frontend, and database, enables simple-command deployment.

- Responsive and Polished UI
 - Finalized CSS with device-reactive breakpoints and consistent color scheme and branding for the FreezeTag identity.
- User Authentication
 - Integrates with backend API keys for user-scoped data access.

3.3.3 Rank 3

- Additional First-Party Plugins
 - UI Extension Plugins
 - Plugin that displays photo locations on the map
 - Owned by Sathya
 - More robust “bulk tag” plugin for large file uploads
 - Owned by Ethan
 - More UI API exposure to support non-pipeline plugins
 - A plugin can add a button/component/page/etc and run custom plugin code based on user input
 - Owned by Brayden
- Architecture-Aware Deployment Support
 - Adds compatibility for ARM-based systems (e.g., M-series Macs).
 - Attempts to utilize available accelerators (GPUs, Metal, etc.)
 - Includes architecture-specific Docker images and build optimizations.
 - Owned by Max

4 Software Engineering Techniques

4.1 Development Process

We believe that Agile will be the best process for our project. We are developing an app that has a lot of features which are cross-dependent on each other, so while we will be planning most features ahead of time, we will need to plan around the specific implementation as we build the software. Relying on an Agile process means that we can be flexible in responding to particulars of any implementation.

4.2 Tools

We are developing in Go and Next.js (which is React-based), both of which have strong support in VSCode. All of us plan on using VSCode because of that support and our familiarity with it.

We plan on generously using libraries that give us desired functionality in a well-implemented way, which is well-supported by using Go Modules for Go code and NPM for Next.js code. Some libraries/tools that we will heavily rely on include Gin, ImageMagick, SQLite, and Docker Compose. Through Gin, we will also get Swagger docs for our backend automatically.

Both the backend and frontend will have strong unit testing. More is specified in Section 4.3, but we intend on enforcing a minimum testing coverage and requiring that unit testing (alongside formatting and linting checks) passes before a branch is merged. Go has unit testing built in, and we are using Jest for testing Next.js code.

Bug tracking, issue tracking, and versioning all will be handled in Gitlab using the provided tooling. This also means we will be using Git specifically for versioning.

4.3 Code Review

We have crafted a set of Contribution Guidelines available on Gitlab. We have summarized/rephrased that text here:

Authors of an MR are expected to perform manual QA/integration testing on their new code. This mostly applies to UI items, but API items can also be tested by hand and it is a good idea to do that as well. Authors of a PR are also expected to write unit tests for any new features they implement and uphold an 80% code coverage minimum.

An MR will be required to be reviewed and approved by a “primary reviewer”, who should be the “next best expert” in that area of the codebase. For example, a frontend MR should be reviewed by at least a frontend developer, if not the frontend developer who most recently worked in that code area. The primary is required to review and approve, all other team members are optional but have 12 hours to leave a review (the 12 hour period can be, and is frequently, waived by a discord “voice vote”). All reviewers can leave a blocking review, but this is disruptive and should be a last-resort. All reviews given should have genuine concerns, and all reviews should be taken seriously, even if the comments are just nitpicks. It is better to change code at the MR stage before it becomes an issue later.

We use an automated pipeline to run formatting checks, linting checks, and unit tests. All stages of the pipeline must pass for a merge to happen, except in emergencies.

4.4 Documentation

Generally, we believe that code should be well-written and self-documenting. We hope that this minimizes the number of unnecessary/redundant inline comments that are liable to becoming stale. Our code review process is designed to help enforce this. We will likely have some portions of our project that will need large, overall documentation, and we will accomplish this primarily by READMEs in the repository. Our intent is for this project to become open source after capstone, and so we don’t want documentation to rely on the Capstone instance wikis which may not be accessible to the public in the future.

4.5 Team Communication and Meetings

We primarily communicate using Discord. This allows us to be flexible, by using text or voice as appropriate, and working synchronously or asynchronously as appropriate or necessary. We all agree that, while it is unrealistic for responses to be instant, that we should be prompt with our responses to help keep up the velocity of the project.

We plan on meeting at least three times a week. We will meet on Monday after Capstone lecture for weekly planning, Wednesday during the regular lecture time for a weekly in-person standup, and Friday morning for our staff meeting with Professor Wood. Other meetings can be planned as needed, and can be in-person or remote as appropriate. Because of our commitment to be communicative over Discord, more regular meetings is most likely not necessary.

5 Timeline

	Ethan Collier	Brayden Jonsson	Max Petersen	Sathya Tadinada
Alpha				
Week 1	Upload progress endpoint	Clean up messy frontend gallery code	Add sorting to the query endpoint	Metadata shown in the image context panel
Week 2	User authentication API endpoints	Upload progress on the frontend	Begin plugin environment development	Fix the search location parser
Week 3	User authentication API endpoints	Docker Compose deployment	Finish plugin environment (to at least a usable state)	User Authentication UI
Week 4	Lock down API endpoints for authentication as configured	Frontend User Authentication Handling	Image Captioning First-Party Plugin	Plugin manager UI
Week 5	Lock down API endpoints for authentication as configured	Frontend User Authentication Handling	Finalize Plugin Manifest/ Plugin Handler Polish	Location Tagging with Map First-Party Plugin
Beta				
Week 6	Plugin Import From Git Repo	Responsive Frontend Design (Mobile Support)	Expose read-only database to plugins	Responsive Frontend Design (Mobile Support)
Week 7	Plugin Import From Git Repo	Third-Party .zip Email Plugin	Give each plugin its own SQLite table	Display errors to users with a common flow
Week 8	Bulk Export First-Party Plugin	Build the marketing website	Facial Recognition	Build the marketing website

			First-Party Plugin	
Week 9	Bulk Tag First-Party Plugin	Settings UI	Facial Recognition First-Party Plugin	Polish marketing website, add basic documentation
Week 10	Bulk Tag First-Party Plugin	Custom UI Plugin Support	Plugin Flows Backend Support	Plugin Flows Frontend Support
Release				
Week 11	Enabling/Disabling Plugins	Custom UI Plugin Support	Plugin Flows Backend Support	Plugin Flows Frontend Support
Week 12	Release-Ready Implementation of First-Party Plugins	Custom UI Plugin Support	System-Aware Optimized Deployment	Photo Locations on Map First-Party Plugin
Week 13	Testing and Bugfixing Backend	Testing and Bugfixing Frontend	Testing and Bugfixing Plugins	Photo Locations on Map First-Party Plugin
Week 14	Test, Bugfix, Polish	Test, Bugfix, Polish	Test, Bugfix, Polish	Test, Bugfix, Polish
Week 15	Test, Bugfix, Polish	Test, Bugfix, Polish	Test, Bugfix, Polish	Test, Bugfix, Polish

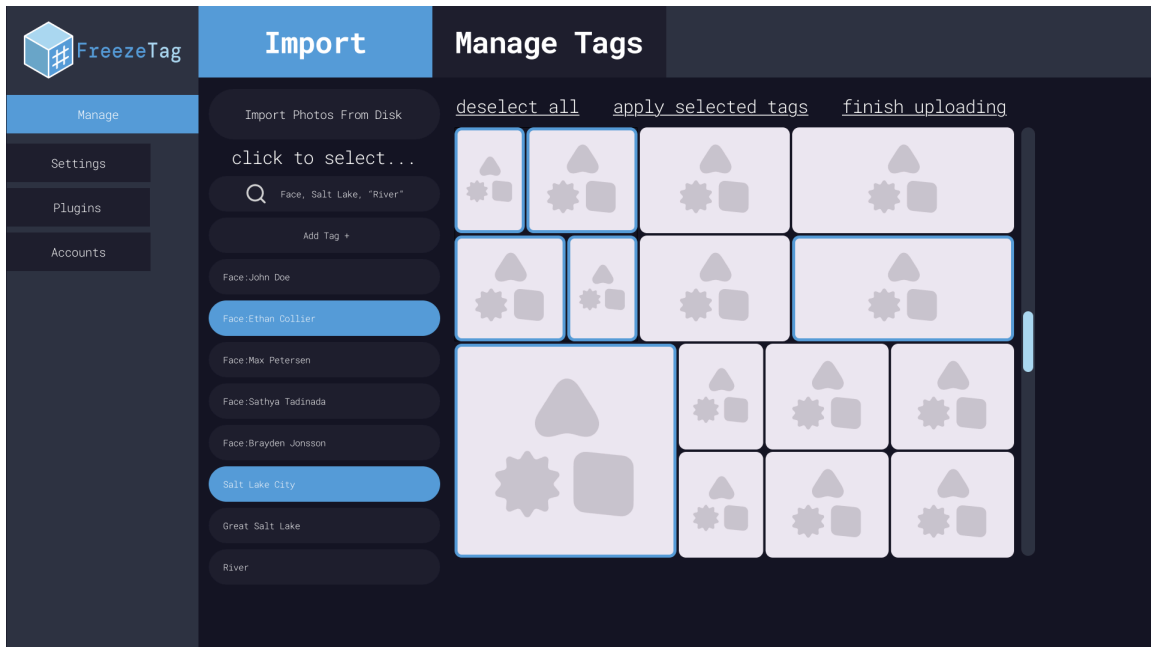
6 Appendix A: UI Sketches

6.1 Gallery Page

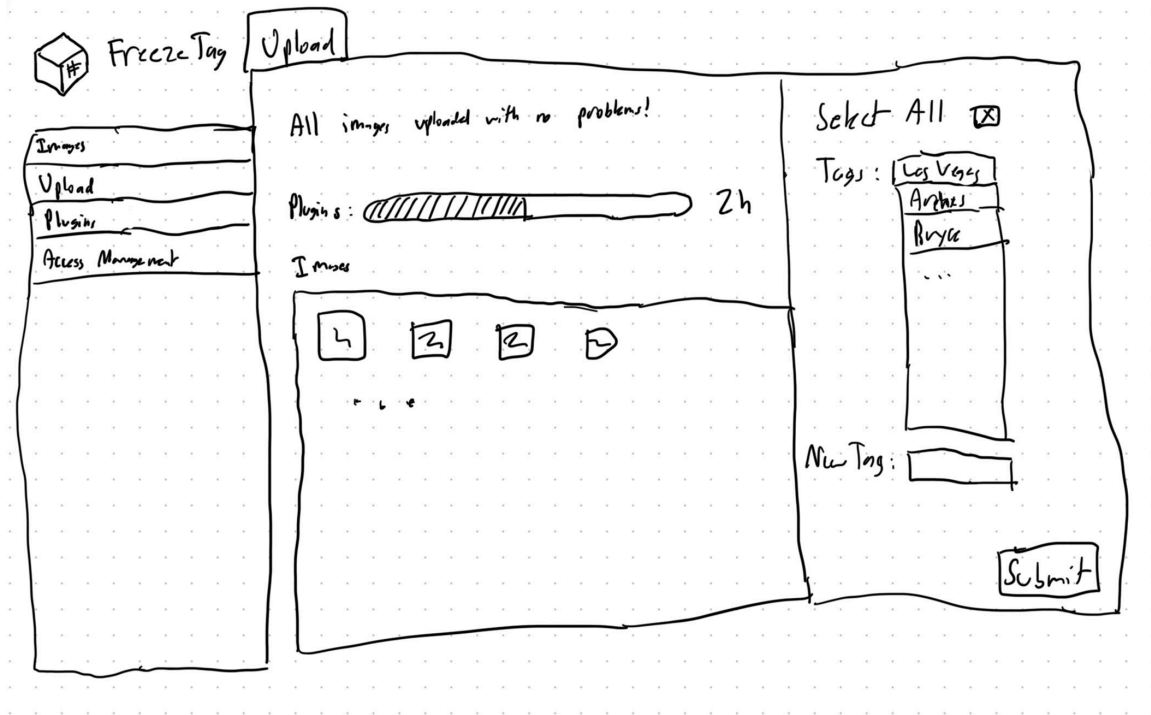


6.2 Import/Upload Page

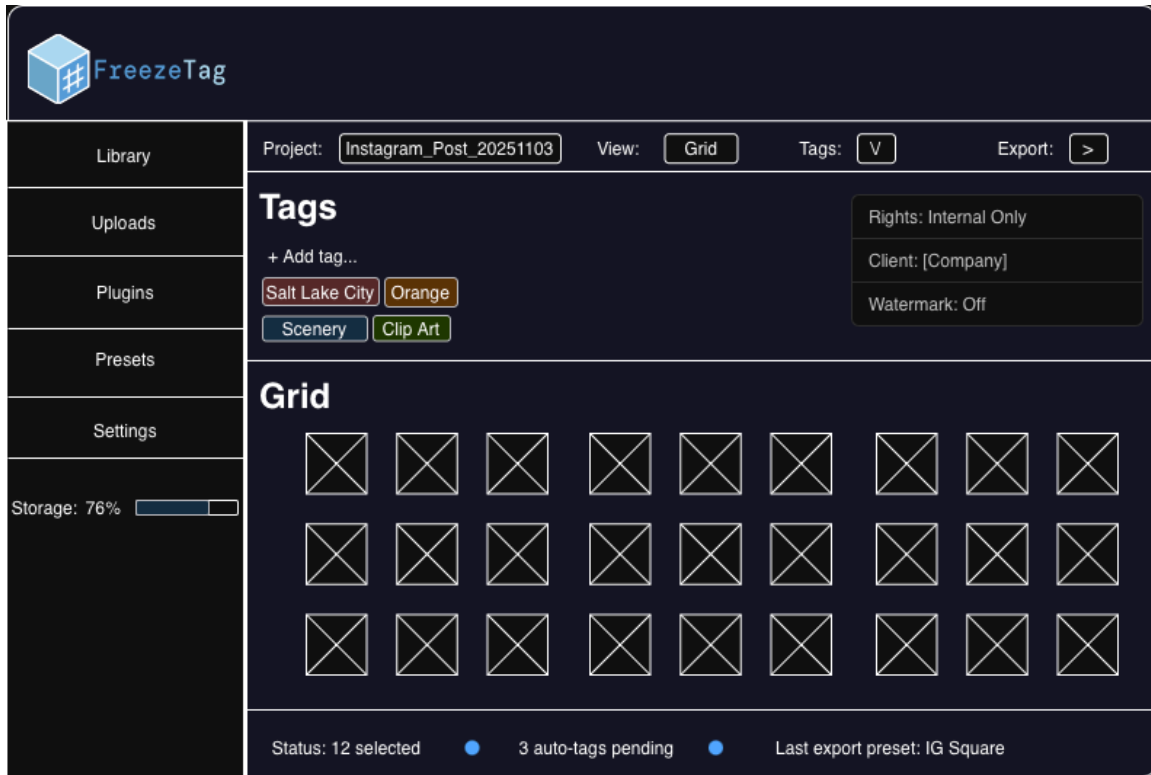
6.2.1



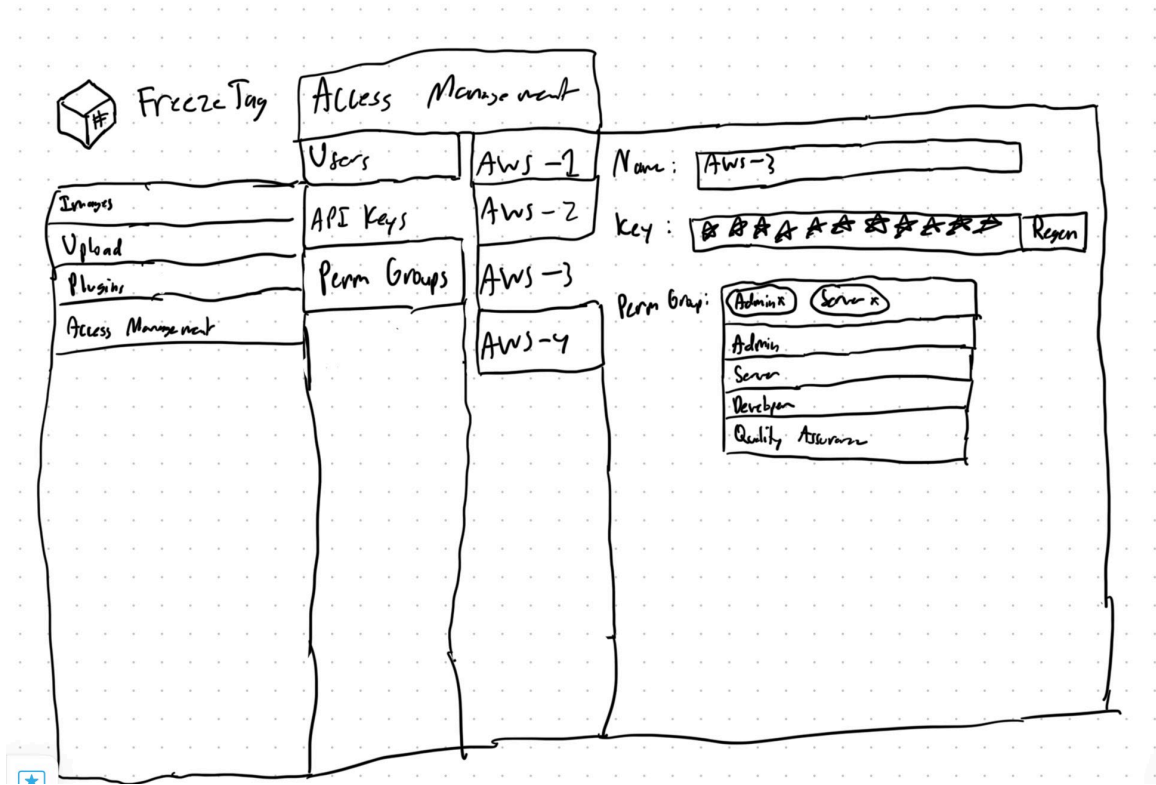
6.2.2



6.3 Export Page



6.4 Admin Page



7 Appendix B: Use Cases

7.1 Use Case 1

Number	Use Case 1
Title	Family Pictures
Preparer	Max Petersen
Actor	Ordinary (Maybe tech-savvy) person
User Story	Whenever I have family get-togethers, we're always taking a lot of photos on our phones. It would be nice if I could easily upload pictures from my phone and look through them in a couple different ways later.
Course of Events	1. Connect to a running FreezeTag instance from my phone's browser 2. Tap the upload button and select photos from my gallery 3. Give the entire upload a global tag (e.g. "Thanksgiving") 4. Finalize the upload so that my pictures are stored. 5. Later, come back and scroll through pictures in chronological order • If I want to view a specific event, I could search for a tag (e.g. "Thanksgiving") in the search bar. • I could even view a specific time frame, e.g. "Thanksgiving, 2025".
Exceptions/Alternatives	• Manual Tags - Aside from the global tag provided during upload, groups of 1 or more photos can be given more tags.
Related UI	See Section 6.1 and Section 6.2

7.2 Use Case 2

Number	Use Case 2
Title	Business
Preparer	Brayden Jonsson
Actor	SRE at a Tech Company
User Story	As an SRE at a document-based SaaS company, I need to find an image storage solution that we can self-host (for security and data integrity reasons) which is easy to interface with our current back-end, like a database.
Course of Events	1. Research the FreezeTag backend API, with our documentation 2. Implement connections to our API in the current company codebase 3. Using the infrastructure that the company already owns or has access to, deploy FreezeTag and access image tagging and plugin features directly through our API 4. Manage API keys, access, and other admin tasks through the user-friendly FreezeTag frontend, which is still accessible. 5. Extend FreezeTag with plugins to automatically delete and remove old, unused images based on tagging data. Use the frontend as an admin panel to control the rules of this plugin.
Exceptions/Alternatives	N/A
Related UI	See Section 6.4 and Section 6.2.2 (plugin progress bar)

7.3 Use Case 3

Number	Use Case 3
Title	Brand Curator
Preparer	Sathya Tadinada
Actor	Graphic Designer
User Story	As a freelance graphic designer, I want to import project assets, apply consistent tags (that keep track/organize into various categories like client, project, usage rights, format, etc.) and quickly find/export the exact variants I need so that hand-offs to my clients are fast and streamlined.
Course of Events	1. Create or open a project/collection for a client or project 2. Bulk-import images/exports (PNG/JPG/WEBP/PSD Previews) 3. Run selected plugins (color extraction, auto object/scene tagging, etc.) to tag the batch 4. Add batch tags (client, project, rights, etc.) and various per-image tags (“Instagram square”, “SVG logo”, etc.) 5. Review and edit tags in the tagging section/panel. 6. Search by tags or fuzzy/natural language to retrieve needed assets with ease 7. Select results and run an export workflow (watermark, sizes/formats/preset naming, etc.) 8. Store the exported package in a handoff folder for clients.
Exceptions/Alternatives	• If GPU/ML plugins are unavailable or cannot be run on-device, proceed with manual/batch tagging instead. • If certain files cannot render a preview, show a default/fallback thumbnail
Related UI	See Section 6.3 and Section 6.2

7.4 Use Case 4

Number	Use Case 4
Title	Client Pictures
Preparer	Ethan Collier
Actor	Professional Photographer
User Story	Recently, I had 2 separate large photoshoots including one of my clients. This client needed the original images of himself sent to them. These photoshoots are large, and as such it is hard to find specific images of him, and it is impractical to send over such a large amount of data.
Course of Events	1. User connects to the FreezeTag instance web UI 2. User has enabled the ML Face ID plugin via a Plugin Manager Menu 3. User enters the name of the person into the Tag Based Search Bar 4. search results generate an Gallery filled with every picture of the client taken in every session 5. Gallery gets further refined with additional tags to ensure that it is only pictures of the client, and no group pictures 6. the Gallery is then Exported to be delivered to the client
Exceptions/Alternatives	• The user may have a plugin enabled that automatically sends the Gallery to the client via email, skipping the middleman device
Related UI	See Section 6.2, Section 6.1, and Section 6.3

7.5 Use Case 5

Number	Use Case 5
Title	Family Guy
Preparer	Max Petersen
Actor	Mother
User Story	I have lots of pictures of my family members, but they are unsorted and uncategorized. I would like to be able to search the pictures by who is in them, without needing to manually tag every picture with their name.
Course of Events	1. Connect to FreezeTag and go to the “Plugins” menu 2. Download/enable the face recognition plugin from the official FreezeTag first party plugins 3. Tag good pictures of each family member’s face with face: [name] 4. Newly uploaded pictures will be scanned for each person with a face: tag, matching people will get their name added as a tag
Exceptions/Alternatives	N/A
Related UI	See Section 6.1 and Section 6.2

8 Appendix C: Revisions

8.1 Introduction

No changes.

8.2 Background

- Changed List of Technologies to List of Leveraged Technologies and added List of Technologies to Implement to clearly delineate what we will leverage through libraries and what we will write.

8.3 Requirements

- Added more information about each of the engineer's roles.
- Defined clear ownership over the current suite of Tier 3 features.
- Added a list of "Primary Assignments" for each engineer.
- More clearly defined Max's primary Tier 3 feature

8.4 Software Engineering Techniques

- Turned the copy of our contribution guidelines into a summary, to fit within the page limit established by the rubric.

8.5 Timeline

No changes.

8.6 Appendix A: UI Sketches

- Moved UI Sketches in front of Use Cases, changed title to include "Appendix".

8.7 Appendix B: Use Cases

- Moved Use Cases behind UI Sketches, changed title to include "Appendix".
- Reformatted use cases to use tables.
- Added Use Case 5 "Family Guy"